DD_CROSS-00 - System Traceability and Implementation Map

Version: v1.0

Status: Draft for review

Owner: Architecture / Delivery governance

Purpose: keep SOPHIX V3 coherent across backend DDs, frontend DDs, APIs, events, data ownership, and end-to-end testing.

This document is not a replacement for module DDs. It is the control map used by dev leads, QA leads, frontend leads, and architects to verify that every user-facing feature has a clear owning frontend, backend module API, event contract, data owner, and test path.

Binding simplification baseline:

- `99_analysis/Sophix_V3_Performance_Complexity_Assessment_20260601.md`
- `99_analysis/Sophix_V3_MVP_Runtime_Simplification_Baseline_20260601.md`

If this map or a module DD seems to imply one deployment per DD, one worker pod per topic, all-to-all synchronous calls, or full first-release implementation of advanced scope, the MVP runtime simplification baseline is authoritative.

1. Why This Document Exists

Individual modules can be implemented correctly and still fail as a system if their boundaries are unclear.

This document prevents that by answering five questions for every important feature:

Question	Required answer
Who uses it?	Frontend app, role, and channel.
Which module owns the command?	One backend module must own the write/API command.
Which modules provide supporting reads?	Panels may read from several modules, but ownership must stay explicit.
Which events connect the modules?	Kafka topics/events and inbox/outbox expectations.
How is it tested end to end?	Scenario test across frontend, APIs, events, and final state.

Rule: a feature is not ready for implementation until this map has an entry for it or the owning DD explicitly declares it out of scope.

2. System Direction

SOPHIX V3 is designed as a modular BSS where:

- foundation services provide shared auth, DB rules, cache rules, Kafka/outbox/inbox, Camunda, Drools, and file storage;
- business modules own their own data and APIs;
- long-running business operations are explicit workflows with statuses, retries, and correlation ids;
- frontend apps call module APIs through the approved gateway/API base URL;
- no frontend app or module reads another module's database directly;
- cross-module integration uses APIs for synchronous needs and events for asynchronous state propagation;
- Kenya/WIK is the first deployment, but operator-specific behavior must be configuration, BPMN, Drools, feature flags, translation, or theme, not a code fork.

3. Backend Module Ownership Map

Bundle	Module/DD family	Owns	Does not own
`01_foundations`	`FOUNDATION_AUTH`	Keycloak/OIDC, shared role names, JWT role claims, service tokens, role guard rules.	Module-specific business permission semantics.

Bundle	Module/DD family	Owns	Does not own
01_foundations	FOUNDATION_DB	DB cluster guidance, schemas, migration rules, partitioning recommendations.	Business tables outside generic DB standards.
01_foundations	FOUNDATION_KAFKA	Topic/event rules, outbox/inbox patterns, idempotent consumers.	Business meaning of events.
01_foundations	FOUNDATION_CAMUNDA	Workflow engine standards, external task/user task guidance.	Business BPMN design inside each flow.
01_foundations	FOUNDATION_DROOLS	Rules engine standards and deployment pattern.	Business rules inside each module.
01_foundations	FOUNDATION_CACHE	Cache conventions, invalidation, reconciliation.	Module ownership of source data.
01_foundations	FOUNDATION_FILE_STORAGE	Authorized file storage access, metadata and storage patterns.	Business attachment lifecycle.
02_framework	SUB-LM-01	Subscription master data and subscription status.	Operation orchestration and billing/fulfillment side effects.
02_framework and 03/04	SUB-WF-*	Subscription lifecycle/MACD operations and workflow orchestration.	Customer profile, invoice generation, field execution.
05_billing	BIL-*	Charging, invoicing, invoice status, tax invoices, payments, notes, adjustments, cycle close, dunning, wallet.	Subscription master state and network fulfillment.
06_fulfillment	FUL-02	New customer order capture/onboarding orchestration.	Customer master ownership, payment ledger ownership, WO lifecycle, subscription master.
06_fulfillment	FUL-03/04/05/07	Activation, restriction, termination, and HomePass lifecycle execution.	Subscription command ownership and billing ledger ownership.
06_fulfillment	FUL-08/09/10	Billing anchor change execution support, equipment-swap service rebinding, and technology-migration technical fulfillment.	Subscription business-operation ownership, OSR stock ownership, WO lifecycle.
06_fulfillment	PROV-INT-01	BSS-side provisioning command dispatch, retry, reconciliation, and force-sync audit.	OSS/network-controller implementation.
07_work_order	WO-01	Work order lifecycle, assignment, execution, finalization, WO evidence links.	Ticket lifecycle, customer master, equipment stock source of truth.
08_osr_equipment	OSR-*	Stock chain, equipment instances, swaps, RMA, equipment lifecycle.	Work order dispatch and customer subscription ownership.
08_osr_equipment	OSR-02/05	Procurement, goods receipt into stock, physical inventory audit, discrepancy investigation, approval/write-off linkage.	Supplier contract management, WO dispatch.
09_catalogs	ILM-CFG-01	Customer service APIs, customer profile/account, notes/interactions, KYC where defined.	Ticket lifecycle and subscription lifecycle.
09_catalogs	RLM-CFG-01	HomePass configuration and serviceability data.	Work order execution and customer order orchestration.
09_catalogs	SIP-01, PLM-CFG-*, BIL-CFG-01	Service, package, tax, wallet, discount, adjustment, equipment type, billable event configuration.	Runtime subscription status, billing transactions, stock movements.
10_other	EM-02	Contractor, staff, team, and agent registry.	WO assignment decisions after selection.
10_other	EM-01	Franchise registry, commercial scope, territory/region linkage, franchise lifecycle.	Sales lead conversion and order capture.
10_other	EM-CFG-03	Business RBAC catalog: roles, permissions, scopes, frontend visibility mapping.	Token issuance and identity-provider operation.
10_other	EM-CFG-04	Approval workflow catalog and reusable approval definitions.	Module-specific business decision execution after approval.
10_other	EM-03	CVM/retention case selection, campaign action queues, churn-risk/retention workflow inputs.	Billing ledger, ticket lifecycle, notification transport.

Bundle	Module/DD family	Owns	Does not own
10_other	FA-01/02/03	Field audit process types for network, customer premises, and equipment/stock audit evidence.	WO dispatch engine, OSR stock source of truth.
10_other	REP-01	Reporting data mart, dashboard APIs, exports, reconciliation read models.	Operational write ownership in source modules.
10_other	CUST-INT-01	Cross-module customer interaction timeline/read model where enabled.	Source module transaction ownership.
10_other	NOT-01, ICN-01	Customer notifications and internal communications/staff notification inbox.	Business decisions that trigger notification.
10_other	TCK-01	Ticket/case lifecycle, SLA, assignment, comments, ticket attachments metadata, ticket-to-WO creation.	Customer master, WO lifecycle, billing/subscription execution.
10_other	ASR-01/02/03/04	Service-assurance ticket/service-request flow satellites on top of TCK.	A separate ticketing backend.
10_other	SALES-01	Leads, field-sales activity, territory assignment, conversion attribution.	Fulfillment order ownership after lead conversion.

4. Frontend Application To Module API Map

Frontend DD	Application	Primary modules it must use	Notes
FE-APP-00	Shared frontend foundation	FOUNDATION_AUTH, runtime config, feature flags, file access, API gateway pattern.	Does not implement business screens. Defines rules all apps must follow.
FE-APP-01	BSS Backoffice Web App	ILM, TCK, SUB-LM, SUB-WF, BIL, FUL, WO, OSR, catalogs, EM, NOT, ICN, foundation admin APIs.	Main internal operational surface. Each area must show module/API ownership.
FE-APP-02	Outdoor Sales / Franchise App	SALES-01, RLM/HomePass, SIP/package, ILM/customer/KYC, FUL-02 order capture, BIL payment/wallet visibility, NOT customer comms, file storage.	SALES-01 owns pre-order lead/territory/assignment/attribution. FUL-02 owns submitted orders.
FE-APP-03	Contractor Technical Mobile App	WO-01, OSR/equipment, EM contractor/staff/team, file storage, ICN, TCK only when a field issue creates/updates a ticket.	Must support offline job execution and evidence upload.
FE-APP-04	Customer Self-Care Frontoffice App	ILM customer/account, SUB-LM/SUB-WF limited self-service, BIL invoice/payment/wallet, TCK support tickets, NOT preferences, file storage.	Customer language must hide internal workflow complexity.
FE-APP-05	Reporting Portal	REP-01 dashboard/export/reconciliation APIs, plus read-only module APIs for operational drill-down.	REP-01 owns reporting data mart and dashboard APIs.
FE-CH-USSD-01	USSD Self-Care Channel	Channel/session adapter using ILM, SUB-LM/SUB-WF allowed self-service, BIL/PAY-GW balance and payment actions, TCK support creation, NOT messages.	USSD must not become a second customer-care backend. Keep only session state, audit logs, and channel-specific menu configuration.

Rule: every screen in an application DD must have a table column named `Module API link`, `Owning module`, or equivalent. If it does not, the DD is not ready for implementation.

5. Feature Traceability Matrix

Feature	Frontend owner	Command owner	Supporting read APIs	Event integration	E2E test required
New customer sale to activation	FE-APP-02, Backoffice monitor in FE-APP-01	FUL-02 order capture	ILM, RLM, SIP, BIL, WO, SUB-LM, FUL-03	Payment events, WO finalized/cancelled, subscription created/activated, activation events.	Customer order submitted, paid, KYC approved, install WO finalized, subscription active, service activated.

Feature	Frontend owner	Command owner	Supporting read APIs	Event integration	E2E test required
Customer 360	FE-APP-01, partial in FE-APP-04	No single command owner; read composition only	ILM, SUB-LM, SUB-WF, BIL, FUL, WO, OSR, TCK, NOT/ICN	Module events feed timelines/audit where available.	One customer page renders with partial failure handling when one module is down.
Ticket creation and support WO	FE-APP-01, FE-APP-04	TCK-01	ILM, SUB-LM, WO, OSR, file storage	TicketCreated, TicketWorkOrderCreated, WO status/finalization events.	Ticket created, WO created from ticket, WO finalized, ticket timeline updated.
Subscription activation	FE-APP-01, from order in FE-APP-02	SUB-WF-ACTIVATE-01 or FUL-02 orchestration depending entry point	SUB-LM, BIL, FUL-03	Subscription operation events, billing charge events, activation events.	Activation preview, command accepted, operation status tracked, final subscription active.
Pause/resume/terminate	FE-APP-01, limited future self-care in FE-APP-04	SUB-WF-PAUSE/RESUME/TERMINATE	SUB-LM, BIL, FUL, NOT	Subscription operation events and fulfillment events.	Command idempotent, impact shown, operation completes or fails with retry path.
Upgrade/downgrade/restrict/suspend/relocate/migrate	FE-APP-01	Relevant SUB-WF-* DD	SUB-LM, SIP, RLM, BIL, WO, FUL	SUB-WF events, billing/fulfillment/WO events as applicable.	Each MACD path validates eligibility, starts operation, and reaches expected final state.
Invoice and payment operations	FE-APP-01, FE-APP-04	BIL-*	SUB-LM, ILM, file storage	Invoice issued, payment applied/reversed, dunning events.	Invoice visible, PDF authorized, payment applied once, balance updates.
Dunning to suspension	FE-APP-01, customer visibility in FE-APP-04	BIL-04 triggers SUB-WF-SUSPEND-NP-01	BIL, SUB-LM, SUB-WF, NOT	Dunning state events, suspension operation events, notifications.	Debt state reaches configured level, suspension starts, restriction applied, customer notified.
Work order dispatch and finalization	FE-APP-01, FE-APP-03	WO-01	EM, ILM-CFG-02, OSR, file storage, ILM/SUB context	WO assigned/finalized/cancelled events.	Dispatcher assigns, contractor executes offline/online, evidence uploads, WO finalized.
Equipment swap/RMA	FE-APP-01, FE-APP-03	OSR-RMA-01 with WO support where needed	OSR-INSTANCE, WO, PLM-CFG-06	Swap/RMA events, equipment lifecycle events, WO events.	Defective equipment replaced, stock movement and instance state are correct.
Catalog setup	FE-APP-01	PLM/SIP/RLM/ILM/BIL config modules	Config module APIs	Config changed events if implemented.	New package/service/HomePass/tax config is valid and used by order/subscription/billing.
Contractor/staff setup	FE-APP-01	EM-02	ILM-CFG-02 tech regions	EM changed events if implemented.	Contractor, main handler, staff roster, skills, regions, and availability created; inactive records not selectable.
Notifications and staff tasks	All relevant apps	NOT-01, ICN-01, plus triggering module	Auth/user profile, owning module details	Module events trigger notifications/staff inbox records.	Staff receives task with deep link; customer notification generated once.
Reporting dashboards	FE-APP-05, embedded summaries in FE-APP-01	REP-01	Event store/read models/module read APIs	Kafka events from all core modules.	Dashboard totals reconcile to source modules for a selected day.
Technology migration	FE-APP-01	SUB-WF-MIGRATION-01 for business operation; FUL-10 for technical cross-technology fulfillment	SUB-LM, RLM, SIP, BIL, WO, OSR, FUL-09, PROV-INT	Subscription migration events, WO events, equipment-swap events, provisioning reconciliation events.	Admin starts HFC-to-GPON cohort migration, target HomePass and package validated, WO/equipment/provisioning complete, subscription remains coherent.
Field audits	FE-APP-01, execution in FE-APP-03	FA-01/02/03 audit process type	WO, OSR, RLM, ILM, EM-02, file storage, TCK where audit creates a case	Audit started/completed events, WO events, discrepancy/approval events.	Audit assignment creates/uses WO when field visit required, evidence uploaded, discrepancy routed to OSR/TCK/approval as appropriate.

Feature	Frontend owner	Command owner	Supporting read APIs	Event integration	E2E test required
CVM retention operations	FE-APP-01, reporting in FE-APP-05	EM-03	ILM, SUB-LM, BIL, TCK, SIP/DIS, NOT, CUST-INT	CVM case events, notification events, discount/campaign events.	At-risk customer appears in retention queue, action taken, outcome captured, timeline/reporting updated.
USSD self-care	FE-CH-USSD-01	Channel adapter delegates each command to the owning module	ILM, SUB-LM/SUB-WF, BIL/PAY-GW, TCK, NOT	Payment callback events, ticket events, subscription-operation events.	Customer checks balance, pays, creates a support request, and receives consistent state with self-care/backoffice.

6. Coherence Corrections From 2026-06-01 Review

The DD set has grown since the first gap reports were written. Treat the following as the current control decision when older documents or analysis files disagree.

Topic	Current decision	Developer impact
EM-02 duplicate names	DD_EM-02-Contractor_and_Staff_Registry-v1.0 is canonical. DD_EM-02-Contractor_Team_Agent_Registry-v1.0 is retained only for historical traceability.	Do not implement two EM-02 services. Build the simplified contractor/staff/capacity model.
ASR vs TCK	ASR-01/02/03/04 are ticket/service-request flow satellites on top of TCK-01.	Do not create a second ticketing engine. ASR owns type-specific workflow/rules only.
Technology migration	SUB-WF-MIGRATION-01 owns the business operation. FUL-10 owns the technical cross-technology fulfillment sequence. WO owns dispatch/execution; OSR owns stock/equipment.	Do not duplicate migration state across modules. Use correlation ids between SUB-WF, FUL-10, WO, OSR, and PROV-INT.
Field audits	FA-01/02/03 are three audit process types, not three separate mandatory deployments.	Implement as one field-audit capability where practical, with audit_type driving rules and workflow.
Reporting	REP-01 is the reporting data mart/API. FE-APP-05 consumes REP-01.	Browsers and dashboards must not query operational DBs directly.
USSD	FE-CH-USSD-01 is a channel adapter over existing self-care/module APIs.	Keep only menu/session/channel audit state locally. Do not create separate subscription, billing, or ticket state.
Approval workflows	EM-CFG-04 defines reusable approval catalog entries. Modules still own their own final business state.	Use Camunda user tasks where approval is long-running; do not put approval state only in frontend screens.
RBAC	EM-CFG-03 maps business roles/permissions/scopes; FOUNDATION_AUTH handles tokens and identity integration.	Frontends should use RBAC catalog for visibility; APIs still enforce server-side permissions.
API and test standards	DD_API-00 and DD_QA-01 are current baseline standards.	Per-module OpenAPI and automated contract/E2E tests remain implementation deliverables.

Older analysis documents may still contain "New DD required" or "future module" rows for items now covered. In that case, this map and the current module DD are authoritative.

7. Remaining Follow-Up Work

Work item	Why it remains	Owner
Generate per-module OpenAPI contracts	DD examples are implementation guidance, not machine-validated API contracts.	Backend leads under DD_API-00.
Automate cross-module E2E tests	DD_QA-01 defines the scenarios; execution scripts still need to be built.	QA/automation lead.
Refresh HLD/process-map PDFs	Older presentation/process documents predate the final DD set.	Architecture documentation owner.
Regenerate PDFs/zip after this hardening pass	Review copies must match patched markdown.	Documentation owner.
Decide optional renames	Some late DD codes are technically correct but may need naming cleanup for user-facing clarity.	Architecture governance.

8. Frontend DD Readiness Checklist

Every frontend DD must answer:

- Which backend module API owns each screen action?
- Which supporting read APIs are used by each panel?
- Which roles from `FOUNDATION_AUTH` can see the screen/action?
- Which feature flags hide or show the screen/action?
- Which commands require `Idempotency-Key`?
- Which errors are expected from the owning module and how are they displayed?
- Which offline behavior exists, if any?
- Which files are uploaded/downloaded through `FOUNDATION_FILE_STORAGE`?
- Which events should the screen react to or poll for?
- Which E2E scenario validates the screen?

9. Backend DD Readiness Checklist

Every backend DD must answer:

- What data does this module own?
- Which APIs does it expose?
- Which roles/service callers can use each API?
- Which events does it publish?
- Which events does it consume?
- Which module APIs does it call synchronously?
- Which idempotency keys are required?
- Which tables are source of truth?
- Which operational metrics and alerts are required?
- Which frontend apps or screens depend on it?

10. Cross-Module Rules

10.1 Ownership

One write owner per business object.

Examples:

- TCK owns ticket status. WO must not update ticket status directly.
- WO owns work order status. TCK may request WO creation but must not mutate WO lifecycle tables.
- SUB-LM owns subscription master state. Billing and fulfillment must not write subscription state directly.
- BIL owns invoice/payment/wallet state. Subscription and fulfillment consume billing outcomes but do not write billing ledgers.
- OSR owns equipment instance and stock state. WO captures field evidence and calls/feeds OSR as defined by the relevant DD.

10.2 Frontend Composition

A screen may compose data from many modules, but each panel must show its source module.

Example: Customer 360 is a composition screen. It reads ILM, SUB-LM, SUB-WF, BIL, FUL, WO, OSR, TCK, NOT, and ICN. It does not create a new Customer 360 database unless a separate read model/reporting DD defines one.

10.3 Events

Every cross-module event must have:

- event name;
- owning producer;
- topic;
- idempotency key;
- correlation id;
- schema version;
- consumer behavior;
- replay/retry behavior.

Events are not a substitute for ownership. They communicate that something happened; they do not allow consumers to become hidden owners of the producer's state.

10.4 Workflow

Camunda owns process progression for workflows that require orchestration, timers, user tasks, or long-running state.

Workers execute tasks. Workers may be consolidated operationally, but each worker/topic contract must remain clear in the DD.

10.5 Rules

Drools owns configurable decision rules where the DD explicitly says rules are operator-configurable.

Simple validation should remain in service code unless the DD says it must be configurable by operator/business users.

11. Runtime Simplification Guardrails

These rules are meant to keep the first implementation practical for a budget-conscious medium deployment.

11.1 Worker Consolidation

Worker names in DDs are logical task contracts. They are not mandatory process, pod, or deployment names.

Implementation rule:

- one worker application may handle multiple low-volume task topics;
- metrics, logs, retries, and dead-letter handling must remain visible per logical topic;
- split into separate worker deployments only when throughput, security, dependency isolation, or operational ownership justifies it.

Recommended v1 grouping:

Worker group	Can contain
Subscription operations worker	SUB-WF lifecycle/MACD service tasks.
Billing operations worker	billing generation, payment application support, dunning jobs, invoice repair jobs.
Field operations worker	WO timers, dispatch tasks, field-audit jobs, evidence post-processing.
Equipment operations worker	OSR stock movements, RMA/swap handlers, procurement receipt handlers.
Integration worker	provisioning, payment gateway callbacks, external reconciliation jobs.
Reporting/CVM worker	reporting aggregation, CVM selection, scheduled export jobs.

11.2 Camunda Use

Use Camunda for long-running workflows that need timers, user tasks, compensation, human approvals, or visible operation state.

Do not use Camunda for:

- simple CRUD;
- deterministic request validation;

- short synchronous API composition;
- pure database reconciliation scans that can run as normal scheduled jobs;
- read-only reporting aggregation.

When a DD mentions a worker for a deterministic job, implement it as a Laravel/Spring scheduled command, queue consumer, or batch job unless the DD explicitly requires BPMN orchestration.

11.3 Drools Use

Use Drools when business users/operators may configure policy without code deployment: eligibility, approval requirement, routing, pricing/discount applicability, dunning thresholds, or audit discrepancy treatment.

Keep these in service code:

- required fields;
- enum validation;
- foreign-key existence checks;
- idempotency checks;
- optimistic-locking and unique-constraint handling;
- simple permission checks already covered by RBAC.

Drools should return decisions and reasons. The owning module still writes the authoritative state.

11.4 Performance Guardrails

Avoid frontend and backend fan-out patterns that become bottlenecks:

- Customer 360 may call several module APIs, but each panel should have timeout/partial-failure behavior.
- Heavy dashboards must use `REP-01`, not live joins against operational modules.
- Current transactional truth must be read from the owning module, not cache, unless the DD explicitly marks a read model as authoritative for that use.
- Event consumers must be idempotent and replay-safe before production data volume increases.
- Large history tables must use the partitioning guidance from `FOUNDATION_DB`.

11.5 First-Release Simplifications

For v1, prefer these simplifications unless a specific operator requirement overrides them:

- implement FA-01/02/03 as one field-audit module with `audit_type`;
- implement ASR-01/02/03/04 as TCK ticket types/workflow satellites;
- keep CVM rule-based first, without an ML platform;
- use one reporting mart/API before creating many specialized analytics services;
- keep USSD as a channel adapter over existing APIs;
- consolidate low-volume workers into a few observable worker apps.

11.6 First-Release Phasing

Phase	Build emphasis
Wave 1	Revenue-critical core: customer/catalog, subscription master/framework, activation, payment, basic invoice/wallet, FUL-02 happy path, installation WO, OSR basics, TCK basic ticket, notifications, Backoffice essentials, contractor install execution, REP-01 minimal dashboards.
Wave 2	Operational hardening: pause/resume/terminate, dunning suspension, WO support/shifting, OSR-RMA/swap, provisioning reconciliation, customer self-care, reporting exports/reconciliation.
Wave 3	Advanced scope: full MACD migration, campaigns/discount complexity, CVM expansion, ASR specialization, procurement/audit, field audits, USSD, mediation/rating, advanced franchise/commercial workflows.

Advanced DDs may exist in the design set before Wave 3. Their existence does not mean they must be implemented at full depth in Wave 1.

12. Minimum E2E Test Pack

Before SOPHIX V3 is considered coherent enough for SIT, these tests must pass:

Test	Modules involved	Success criteria
New WIK customer activation	FE-APP-02/01, ILM, RLM, SIP, FUL-02, BIL, WO, SUB-LM/SUB-WF, FUL-03, NOT/ICN	Subscription active, billing correct, WO finalized, activation complete, customer notified.
Payment idempotency	FE-APP-01/04, BIL-01-PAY-01, Kafka/outbox	Same payment reference cannot double-credit. Retry returns same outcome or safe conflict.
Ticket to support WO	FE-APP-01/04, TCK, WO, EM, OSR, NOT/ICN	Ticket creates WO, WO finalization updates ticket timeline, SLA remains visible.
Dunning suspension	BIL-04, SUB-WF-SUSPEND-NP, FUL-04, NOT	Eligible overdue subscription is restricted/suspended and notified.
Relocation	FE-APP-01, SUB-WF-RELOCATION, RLM, WO, OSR, BIL	Target HomePass validated, work executed, equipment/subscription state coherent.
Equipment swap	FE-APP-01/03, WO, OSR-RMA, OSR-INSTANCE, PLM-CFG-06	Old and new serial states correct, stock movement recorded, customer service view updated.
Customer 360 partial failure	FE-APP-01, all read modules	Page renders available panels and clear error state for failed module.
Reporting reconciliation	FE-APP-05, reporting backend, BIL/FUL/WO/TCK events	Dashboard totals reconcile to source module queries for same period.

13. Implementation Governance

Use this sequence:

1. Finish missing gap DDs or explicitly mark them deferred.
2. Create API catalog/OpenAPI contracts for modules used by the next frontend DD.
3. Write frontend DD screen flows with module API links.
4. Add or update E2E scenarios in this document.
5. Implement backend module API and contract tests.
6. Implement frontend screen using the documented API.
7. Run E2E scenario before declaring the feature complete.

Do not allow a squad to implement a screen or command if the owning module API is not identified.

14. Current Status

Area	Status
Core backend module DDs	Broadly coherent and implementable after the 2026-06-01 coherence corrections.
Backoffice frontend	Coherent after module API links were added; continue to patch screens only when new backend APIs change.
Outdoor sales/franchise frontend	Written; uses SALES-01, EM-01, RLM, ILM, SIP, FUL-02, BIL/PAY-GW, and file APIs.
Contractor mobile frontend	Written; uses WO, OSR, EM-02, file storage, ICN, and TCK where field issues create/update tickets.
Customer self-care frontend	Written; uses ILM, SUB, BIL/PAY-GW, TCK/ASR, NOT, and file APIs.
Reporting frontend	Written; uses REP-01, with operational drill-down through read-only module APIs.
USSD channel	Written; implemented as channel adapter over self-care/module APIs.
Cross-module testing	Strategy exists in QA-01; executable automation still pending.